

# claude-windows / 7dc3da51-af38-47b6-83f6-fdda83b60664

## Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\subagents\workflows\wf\_6c18d403-385\agent-ac1ab5d8df47b38fc.jsonl`
- SHA-256: `c334e0b11bdc3b3d92cb2d63e1df8d67a852081a115c655603c7797b72ba9d75`
- Source modified: `2026-07-06T21:01:47+00:00`
- Imported at: `2026-07-08T16:00:07+00:00`
- project: `wf\_6c18d403-385`
- session\_id: `7dc3da51-af38-47b6-83f6-fdda83b60664`

## Transcript

### 1. user (2026-07-06T20:56:00.635Z)

Reviewing GitHub PR #50 of the repo at C:/Users/avidu/Projects/Telegram ? "feat(policy): add F1 pipeline primitives" (branch feat/f1-policy-engine -> main).

It is the first F1 slice of ADR 0012 (policy pipeline, just merged as #40). +280/-14, 4 files:

- src/tg\_video\_filter/policy.py (NEW): PolicyContext, StageResult (+ pass\_/fail helpers), PolicyStage Protocol, evaluate\_pipeline().
- src/tg\_video\_filter/db.py: NEW load\_policy\_config/save\_policy\_config (raw JSON dict over policies.config\_json);  
load\_filter\_config now uses model\_validate(\_policy\_config\_json\_to\_dict(...)) instead of model\_validate\_json;  
save\_filter\_config now MERGES cfg.model\_dump(mode='json') into the existing raw config dict (preserving unknown keys) instead of replacing the whole blob with cfg.model\_dump\_json().
- tests/test\_db.py + tests/test\_policy.py: primitives tests + a preserve-unknown-keys regression test.

A checked-out copy of the PR is at C:/Users/avidu/Projects/\_wt-pr50 (read files there or via 'git -C C:/Users/avidu/Projects/Telegram show origin/feat/f1-policy-engine:<path>').

Run repros with: PYTHONPATH=C:/Users/avidu/Projects/\_wt-pr50/src  
C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe <script> (build schema via db.\_apply\_schema on aiosqlite ":memory:").

BACKGROUND ? why this PR exists: ADR 0012 review found that load\_filter\_config/save\_filter\_config would ERASE future pipeline keys (topics/security\_checks/deep\_inspect) when a user runs /set, because save re-serialized via FilterConfig (which drops unknown keys). F1 fixes that persistence boundary. The main reviewer already verified the happy path works (pipeline keys survive a /set; flat fields update). Your job: find edge cases / bugs / regressions the happy path missed.

Ground rules:

- READ actual code, cite file:line. Run repros. Authoritative gate ALREADY RUN: ruff clean, 501 passed @ 96.63%.
- Report ONLY defects in THIS PR's diff. No praise. Concrete failure scenario per finding. Rank most-severe first.
- Severity: high (real bug/regression/data-loss), medium (edge case, latent, asymmetry), low (typing/hygiene/scope-note).

ADVERSARIALLY VERIFY this finding ? try to REFUTE it. Read the cited code, run a repro with PYTHONPATH=C:/Users/avidu/Projects/\_wt-pr50/src  
C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe.  
CONFIRMED only if the defect genuinely ships in THIS PR's diff and produces the described wrong behaviour. REFUTED if

handled / can't occur / pre-existing / out-of-scope-and-disclosed. PLAUSIBLE if unsettled.  
Corrected severity.  
FINDING (persistence): load\_filter\_config exception type changed from pydantic ValidationError to ValueError/JSONDecodeError on corrupt config\_json  
severity low | src/tg\_video\_filter/db.py:1759  
summary: The new load path (model\_validate over \_policy\_config\_json\_to\_dict) raises builtins.ValueError (non-object JSON) or json.decoder.JSONDecodeError (malformed JSON), whereas the old load\_filter\_config used FilterConfig.model\_validate\_json which raised pydantic ValidationError for both cases.  
scenario: A policies.config\_json holds a non-object value (e.g. the JSON string "not-an-object") or malformed JSON. Old code: load\_filter\_config raised pydantic\_core.ValidationError. New code (verified via repro): raises builtins.ValueError ('policy config\_json must decode to a JSON object') or json.decoder.JSONDecodeError. Any future caller that did `except ValidationError` around load\_filter\_config would silently fail to catch it. No such caller exists today (manager.py:140, commands.py:78/82/112, command\_service.py, bootstrap.py all propagate; no test narrows on ValidationError from this function), so the change is latent only.  
evidence: Repro (r1.py) output: load\_filter\_config on json.dumps('not-an-object') -> builtins.ValueError; on '{not valid json}' -> json.decoder.JSONDecodeError; whereas FilterConfig.model\_validate\_json(...) -> pydantic\_core.ValidationError for both. db.py:1759 uses FilterConfig.model\_validate(\_policy\_config\_json\_to\_dict(policy.config\_json)); \_policy\_config\_json\_to\_dict raises ValueError at db.py:1301 and json.loads raises JSONDecodeError at db.py:1299.  
rec: Acceptable as-is since no caller relies on the exception type. If a stable public contract is desired, wrap the decode/validate in load\_filter\_config and re-raise a single documented exception type (or note the type change in the docstring).

## 2. assistant (2026-07-06T20:56:03.126Z)

I'll adversarially verify this finding by reading the actual code and running a repro.

## 3. user (2026-07-06T20:56:04.001Z)

```

1280
1281     async def get_default_policy(conn: aiosqlite.Connection, account_id: int) -> Policy |
None:
1282         """Return the account's default policy (`is_default=1`)."""
1283         conn.row_factory = aiosqlite.Row
1284         async with conn.execute(
1285             "SELECT * FROM policies WHERE account_id = ? AND is_default = 1",
1286             (account_id,),
1287         ) as cur:
1288             row = await cur.fetchone()
1289             return _row_to_policy(row) if row else None
1290
1291
1292     def _policy_config_json_to_dict(raw: str) -> dict[str, object]:
1293         """Decode a policy config JSON object.
1294
1295         ADR 0012 keeps future policy keys in `policies.config_json`. The value is
1296         expected to be a JSON object so legacy flat `FilterConfig` keys and future
1297         nested stage config can coexist.
1298         """
1299         decoded = json.loads(raw)
1300         if not isinstance(decoded, dict):
1301             raise ValueError("policy config_json must decode to a JSON object")
1302         return decoded
1303
1304
1305     def _policy_config_dict_to_json(config: dict[str, object]) -> str:
1306         """Encode policy config deterministically and compactly."""
1307         return json.dumps(config, separators=(",", ":"), sort_keys=True)
1308
1309
1310     async def load_policy_config(conn: aiosqlite.Connection, account_id: int) -> dict[str,
object]:

```

```

1311         """Load the raw default policy config as a JSON object.
1312
1313         Unlike :func:`load_filter_config`, this preserves future ADR 0012 keys such
1314         as ``topics`` / ``security_checks`` / ``deep_inspect`` instead of validating
1315         through the flat ``FilterConfig`` model.
1316         """
1317         policy = await get_default_policy(conn, account_id)
1318         if policy is None:
1319             return FilterConfig().model_dump(mode="json")
1320         return _policy_config_json_to_dict(policy.config_json)
1321
1322
1323     async def save_policy_config(
1324         conn: aiomysql.Connection, account_id: int, config: dict[str, object]
1325     ) -> None:
1326         """Persist the raw default policy config object."""
1327         existing = await get_default_policy(conn, account_id)
1328         await upsert_policy(
1329             conn,
1330             Policy(
1331                 id=existing.id if existing is not None else None,
1332                 account_id=account_id,
1333                 name="default",
1334                 config_json=_policy_config_dict_to_json(config),
1335                 is_default=True,
1336                 created_at=(existing.created_at if existing is not None else _now()),
1337                 updated_at=_now(),
1338             ),
1339         )
1340
1341
1342     # --- Destination / RoutingRule / DeliveryAttempt repos (ADR 0011, Track C) --
1343
1344
1345     async def upsert_destination(conn: aiomysql.Connection, dest: Destination) ->
Destination:
1346         """Insert or update a destination row by (account, provider, target)."""
1347         await conn.execute(
1348             """INSERT INTO destinations
1349             (account_id, provider, target, kind, title, is_output, created_at)
1350             VALUES (?, ?, ?, ?, ?, ?, ?)
1351             ON CONFLICT(account_id, provider, target) DO UPDATE SET
1352             kind=excluded.kind,
1353             title=excluded.title,
1354             is_output=excluded.is_output""",
1355             (
1356                 dest.account_id,
1357                 dest.provider,
1358                 dest.target,
1359                 dest.kind,
1360                 dest.title,
1361                 int(dest.is_output),
1362                 dest.created_at.isoformat(),
1363             ),
1364         )
1365         await conn.commit()
1366         conn.row_factory = aiomysql.Row
1367         async with conn.execute(
1368             """SELECT * FROM destinations
1369             WHERE account_id = ? AND provider = ? AND target = ?""",

```

#### 4. assistant (2026-07-06T20:56:08.635Z)

The finding acknowledges the exception type change but rates it low because no caller relies on the old type. Let me verify both claims: (1) the exception type actually changes, and (2) no

caller narrows on ValidationError. Let me check the old code on main and the callers.

## 5. user (2026-07-06T20:56:11.290Z)

```
1698:async def load_filter_config(conn: aiosqlite.Connection, user_id: int) -> FilterConfig:
1708:    return FilterConfig.model_validate_json(policy.config_json)
1711:async def save_filter_config(conn: aiosqlite.Connection, user_id: int, cfg: FilterConfig)
-> None:
```

## 6. user (2026-07-06T20:56:11.390Z)

```
1740        matched=item.matched,
1741        forwarded=False,
1742        seen_at=item.seen_at,
1743    )
1744
1745
1746 # --- filter config shims (backwards-compatible view over policies) -----
1747
1748
1749 async def load_filter_config(conn: aiosqlite.Connection, user_id: int) -> FilterConfig:
1750     """Deprecated shim: load the account's default :class:`Policy` as FilterConfig.
1751
1752     ADR 0011 (Option 2A): ``filter_config`` is renamed to
1753     ``filter_config_deprecated``; this reads ``policies WHERE is_default=1``
1754     instead. Falls back to :data:`FilterConfig` defaults if no policy exists.
1755     """
1756     policy = await get_default_policy(conn, user_id)
1757     if policy is None:
1758         return FilterConfig()
1759     return FilterConfig.model_validate(_policy_config_json_to_dict(policy.config_json))
1760
1761
1762 async def save_filter_config(conn: aiosqlite.Connection, user_id: int, cfg:
FilterConfig) -> None:
1763     """Deprecated shim: update flat ``FilterConfig`` keys on the default policy.
1764
1765     ADR 0012: future pipeline keys live in the same ``policies.config_json``
1766     object. Do not overwrite the whole blob with ``cfg.model_dump_json()``;
1767     merge the flat config fields into the existing object so unknown policy
1768     keys survive legacy ``/set`` command updates.
1769     """
1770     existing = await get_default_policy(conn, user_id)
1771     raw_config: dict[str, object] = {}
1772     if existing is not None:
1773         try:
1774             raw_config = _policy_config_json_to_dict(existing.config_json)
1775         except (json.JSONDecodeError, ValueError):
1776             raw_config = {}
1777     raw_config.update(cfg.model_dump(mode="json"))
1778     await save_policy_config(conn, user_id, raw_config)
1779
```

## 7. assistant (2026-07-06T20:56:14.821Z)

Confirmed the code change. Old line 1708:  
`FilterConfig.model\_validate\_json(policy.config\_json)`. New line 1759:  
`FilterConfig.model\_validate(\_policy\_config\_json\_to\_dict(policy.config\_json))`.

Note `save\_filter\_config` catches both `json.JSONDecodeError` and `ValueError` ? so on the save path the type change is handled. But `load\_filter\_config` does NOT catch; it propagates. Let me verify the exception types via repro and check callers.

## 8. user (2026-07-06T20:56:17.168Z)

```

C:/Users/avidu/Projects/_wt-pr50/src/tg_video_filter/commands.py:78:         current_cfg = await
db.load_filter_config(conn, user.id)
C:/Users/avidu/Projects/_wt-pr50/src/tg_video_filter/commands.py:82:         new_cfg = await
db.load_filter_config(conn, user.id)
C:/Users/avidu/Projects/_wt-pr50/src/tg_video_filter/commands.py:112:     cfg = await
db.load_filter_config(conn, user_id)
C:/Users/avidu/Projects/_wt-pr50/src/tg_video_filter/db.py:619:     #      (Option 2A).
is_default=1 so the legacy load_filter_config shim reads
C:/Users/avidu/Projects/_wt-pr50/src/tg_video_filter/db.py:1313:     Unlike
:func:`load_filter_config`, this preserves future ADR 0012 keys such
C:/Users/avidu/Projects/_wt-pr50/src/tg_video_filter/db.py:1864:     "load_filter_config",
C:/Users/avidu/Projects/_wt-pr50/src/tg_video_filter/manager.py:140:         cfg = await
db.load_filter_config(conn, session.account_id)
C:/Users/avidu/Projects/_wt-pr50/src/tg_video_filter/models.py:193:     ``load_filter_config`` /
``save_filter_config`` shims read/write that row.
Binary file C:/Users/avidu/Projects/_wt-
pr50/src/tg_video_filter/__pycache__/commands.cpython-313.pyc matches
Binary file C:/Users/avidu/Projects/_wt-pr50/src/tg_video_filter/__pycache__/db.cpython-313.pyc
matches
Binary file C:/Users/avidu/Projects/_wt-
pr50/src/tg_video_filter/__pycache__/manager.cpython-313.pyc matches
Binary file C:/Users/avidu/Projects/_wt-
pr50/src/tg_video_filter/__pycache__/models.cpython-313.pyc matches

```

### 9. assistant (2026-07-06T20:56:20.971Z)

No `ValidationError` reference anywhere in src. Let me confirm no caller wraps  
`load\_filter\_config` in an `except` that narrows on `ValidationError`, and run the repro to  
confirm the exception types.

### 10. user (2026-07-06T20:56:23.066Z)

```

C:/Users/avidu/Projects/_wt-pr50/tests/test_db.py:527:     """A config with min_duration >
max_duration must raise ValidationError."""
C:/Users/avidu/Projects/_wt-pr50/tests/test_db.py:528:     from pydantic import ValidationError
C:/Users/avidu/Projects/_wt-pr50/tests/test_db.py:532:     with pytest.raises(ValidationError,
match="min_duration_s.*must be <="):
C:/Users/avidu/Projects/_wt-pr50/tests/test_db.py:603:     from pydantic import ValidationError
C:/Users/avidu/Projects/_wt-pr50/tests/test_db.py:607:     with pytest.raises(ValidationError):
C:/Users/avidu/Projects/_wt-pr50/src/tg_video_filter/commands.py:130:     except (OSError,
TimeoutError, RPCError):
C:/Users/avidu/Projects/_wt-pr50/src/tg_video_filter/manager.py:85:         #
gather.return_exceptions=True so one session's failure doesn't tear
C:/Users/avidu/Projects/_wt-pr50/src/tg_video_filter/manager.py:87:         results = await
asyncio.gather(*tasks, return_exceptions=True)
C:/Users/avidu/Projects/_wt-pr50/src/tg_video_filter/manager.py:88:         except
asyncio.CancelledError:
C:/Users/avidu/Projects/_wt-pr50/src/tg_video_filter/manager.py:92:         await
asyncio.gather(*tasks, return_exceptions=True)

```

### 11. assistant (2026-07-06T20:56:26.582Z)

There are two `pytest.raises(ValidationError)` tests in test\_db.py (lines 527-532 and 603-607).  
These are important ? if either of these tests exercises `load\_filter\_config` on corrupt config,  
the exception type change would break them. But the finding claims 501 tests pass, so let me  
check what these tests actually do. Let me read them and run the repro.

### 12. user (2026-07-06T20:56:27.126Z)

```

520         for mode in DeliveryMode:
521             await db.save_filter_config(conn, USER_ID, FilterConfig(delivery_mode=mode))
522             loaded = await db.load_filter_config(conn, USER_ID)
523             assert loaded.delivery_mode == mode
524

```

```

525
526 async def test_filter_config_rejects_inverted_duration_bounds() -> None:
527     """A config with min_duration > max_duration must raise ValidationError."""
528     from pydantic import ValidationError
529
530     from tg_video_filter.models import FilterConfig
531
532     with pytest.raises(ValidationError, match="min_duration_s.*must be <="):
533         FilterConfig(min_duration_s=7200, max_duration_s=5)
534
535
536 def test_filter_config_accepts_equal_min_max_duration() -> None:
537     """min_duration_s == max_duration_s is valid (exact-match filter)."""
538     from tg_video_filter.models import FilterConfig
539
540     cfg = FilterConfig(min_duration_s=60, max_duration_s=60)
541     assert cfg.min_duration_s == 60
542     assert cfg.max_duration_s == 60
543
544
545 async def test_old_config_without_delivery_mode_defaults_to_forward(
546     conn: aiosqlite.Connection,
547 ) -> None:
548     """A pre-v1.1 config JSON (no delivery_mode key) must default to FORWARD."""
549     await _seed_user(conn)
550     # Simulate an old row by inserting raw JSON without the delivery_mode field
551     # into the default policy (ADR 0011: filter_config is now policies).
552     old_json =
553     '{"min_duration_s":5,"max_duration_s":7200,"target_height":1080,"max_size_mb":2048}'
554     await conn.execute(
555         "INSERT OR REPLACE INTO policies (account_id, name, config_json, is_default, "
556         "created_at, updated_at) VALUES (?, 'default', ?, 1, CURRENT_TIMESTAMP, "
557         "CURRENT_TIMESTAMP)",
558         (USER_ID, old_json),
559     )
560     await conn.commit()
561     loaded = await db.load_filter_config(conn, USER_ID)
562     from tg_video_filter.models import DeliveryMode
563
564     assert loaded.delivery_mode == DeliveryMode.FORWARD
565
566
567 async def test_delivery_target_round_trips(conn: aiosqlite.Connection) -> None:
568     """The delivery_target field survives the JSON round trip for all forms."""
569     await _seed_user(conn)
570     for target in ("me", "@my-feed", "-1001234567890", "1234567890"):
571         await db.save_filter_config(conn, USER_ID, FilterConfig(delivery_target=target))
572         loaded = await db.load_filter_config(conn, USER_ID)
573         assert loaded.delivery_target == target
574
575
576 async def test_old_config_without_delivery_target_defaults_to_me(
577     conn: aiosqlite.Connection,
578 ) -> None:
579     """A pre-v1.2 config JSON (no delivery_target key) must default to 'me'."""
580     await _seed_user(conn)
581     old_json = (
582         '{"min_duration_s":5,"max_duration_s":7200,"target_height":1080,'
583         '"max_size_mb":2048,"delivery_mode":"forward"}'
584     )
585     await conn.execute(
586         "INSERT OR REPLACE INTO policies (account_id, name, config_json, is_default, "
587         "created_at, updated_at) VALUES (?, 'default', ?, 1, CURRENT_TIMESTAMP, "
588         "CURRENT_TIMESTAMP)",
589         (USER_ID, old_json),

```

```

587         )
588         await conn.commit()
589         loaded = await db.load_filter_config(conn, USER_ID)
590         assert loaded.delivery_target == "me"
591
592
593     def test_filter_config_coerces_int_delivery_target() -> None:
594         """An int chat id is coerced to str (so JSON stays uniform)."""
595         from tg_video_filter.models import FilterConfig
596
597         cfg = FilterConfig(delivery_target=-1001234567890) # type: ignore[arg-type]
598         assert cfg.delivery_target == "-1001234567890"
599
600
601     def test_filter_config_rejects_empty_delivery_target() -> None:
602         """An empty delivery_target is invalid (min_length=1)."""
603         from pydantic import ValidationError
604
605         from tg_video_filter.models import FilterConfig
606
607         with pytest.raises(ValidationError):
608             FilterConfig(delivery_target="")
609
610
611     async def test_pragmas_are_set_on_init(conn: aiosqlite.Connection) -> None:
612         """Verify WAL mode, foreign keys, and busy timeout are enabled."""
613         await db._apply_schema(conn)
614         async with conn.execute("PRAGMA journal_mode") as cur:

```

### 13. assistant (2026-07-06T20:56:30.108Z)

Both `ValidationError` tests exercise `FilterConfig(...)` constructor directly, not `load\_filter\_config` on corrupt data. So neither test is affected by this change. The finding's claim holds ? no caller or test narrows on `ValidationError` from `load\_filter\_config`.

Now let me run the repro to confirm the exception types actually differ.

### 14. user (2026-07-06T20:58:38.889Z)

Command running in background with ID: bqgj7yb1v. Output is being written to: C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\tasks\bqgj7yb1v.output. You will be notified when it completes. To check interim output, use Read on that file path.

### 15. user (2026-07-06T20:58:46.122Z)

Traceback (most recent call last):

```

File "C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\scratchpad\r1.py", line 29, in <module>
    asyncio.run(main())
    ~~~~~^~~~~~
File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\asyncio\runners.py", line 195, in run
    return runner.run(main)
    ~~~~~^~~~~~
File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\asyncio\runners.py", line 118, in run
    return self._loop.run_until_complete(task)
    ~~~~~^~~~~~
File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\asyncio\base_events.py", line 725, in run_until_complete
    return future.result()
    ~~~~~^~~~~~
File "C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-

```

```
Indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\scratchpad\r1.py", line 10, in main
    await conn.execute("INSERT INTO accounts (id, provider, external_id, created_at) VALUES
(1, 'tg', 'x', CURRENT_TIMESTAMP)")
File "C:\Users\avidu\Projects\Telegram\.env\Lib\site-packages\aiosqlite\core.py", line 223,
in execute
    cursor = await self._execute(self._conn.execute, sql, parameters)
                ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
File "C:\Users\avidu\Projects\Telegram\.env\Lib\site-packages\aiosqlite\core.py", line 160,
in _execute
    return await future
           ^^^^^^^^^^^
File "C:\Users\avidu\Projects\Telegram\.env\Lib\site-packages\aiosqlite\core.py", line 63, in
_connection_worker_thread
    result = function()
             ^^^^^^^^^
sqlite3.OperationalError: table accounts has no column named external_id
```

Schema differs. Let me check the accounts table schema and simplify ? insert the policy directly without needing an account row (get\_default\_policy just queries policies by account\_id).

Command running in background with ID: b2w12p0ql. Output is being written to:  
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\tasks\b2w12p0ql.output. You will be notified when it completes. To check interim output, use Read on that file path.

```

Traceback (most recent call last):
  File "C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\scratchpad\r2.py", line 24, in <module>
    asyncio.run(main())
    ~~~~~^~~~~~
  File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\asyncio\runners.py", line 195, in run
    return runner.run(main)
    ~~~~~^~~~~~
  File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\asyncio\runners.py", line 118, in run
    return self._loop.run_until_complete(task)
    ~~~~~^~~~~~
  File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\asyncio\base_events.py", line 725, in run_until_complete
    return future.result()
    ~~~~~^
  File "C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\scratchpad\r2.py", line 10, in main
    await conn.execute(
        "INSERT OR REPLACE INTO policies
(account_id,name,config_json,is_default,created_at,updated_at) "
        "VALUES (1,'default',?,1,CURRENT_TIMESTAMP,CURRENT_TIMESTAMP)", (badjson,))
  File "C:\Users\avidu\Projects\Telegram\.venv\Lib\site-packages\aiosqlite\core.py", line 223, in execute
    cursor = await self._execute(self._conn.execute, sql, parameters)
    ~~~~~^~~~~~
  File "C:\Users\avidu\Projects\Telegram\.venv\Lib\site-packages\aiosqlite\core.py", line 160, in _execute
    return await future
    ~~~~~^~~~~~
  File "C:\Users\avidu\Projects\Telegram\.venv\Lib\site-packages\aiosqlite\core.py", line 63, in _connection_worker_thread
    result = function()

```





- non-object JSON: NEW raises ``builtins.ValueError``, OLD raised ``pydantic_core.ValidationError``
- malformed JSON: NEW raises ``json.decoder.JSONDecodeError``, OLD raised ``pydantic_core.ValidationError``

Now the verdict assessment. The finding is factually correct: the exception type genuinely changed in this PR's diff (db.py:1759). However, the finding itself already correctly assesses the impact as latent-only:

- No ``ValidationError`` reference exists anywhere in ``src/``.
- The three callers (commands.py:78/82/112, manager.py:140) all propagate without narrowing.
- The two ``pytest.raises(ValidationError)`` tests exercise the ``FilterConfig(...)`` constructor directly, not ``load_filter_config`` on corrupt data, so they are unaffected.
- 501 tests pass confirms no test regression.

This is a real, verified behavioral change but with no current wrong behavior ? no caller relies on the old exception type, so nothing breaks today. It's a latent contract change. The finding's own severity of "low" is appropriate. Given the change genuinely ships and produces a different (documented-as-plausible) exception type, but causes no observable wrong behavior today, this sits at CONFIRMED-as-a-fact but the "wrong behaviour" is purely latent. Per the rubric, CONFIRMED requires the defect to "produce the described wrong behaviour" ? the described behavior (exception type change) does occur, but it's latent. The finding accurately scopes it as latent. I'll mark CONFIRMED at low severity since the type change is real and verified, matching the finding's own framing.

## **26. user (2026-07-06T21:01:47.684Z)**

Structured output provided successfully